

SEARCHING AND SORTING

FOURTH-WEEK ASSIGNMENT FOR THE DATA STRUCTURE



Lecturer :

Andi Iwan Nurhidayat, S.Kom., M.T.

Presented by :

Aisyah Currents

25091397108 / 2025I

INFORMATIC MANAGEMENT MAJOR

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2026

PRACTICUM

Assignment 1: Modified Binary Search

Implement the function `countOccurrences(sortedList, target)` which counts how many times a value appears in a sorted list. The function must run in $O(\log n)$ time complexity, not $O(n)$. Hint: use binary search twice — once to find the left boundary (first occurrence) and once for the right boundary (last occurrence).

Example:

```
countOccurrences([1, 2, 4, 4, 4, 4, 7, 9, 12], 4) -> 4
countOccurrences([1, 2, 4, 4, 4, 4, 7, 9, 12], 5) -> 0
```

The Source Code:

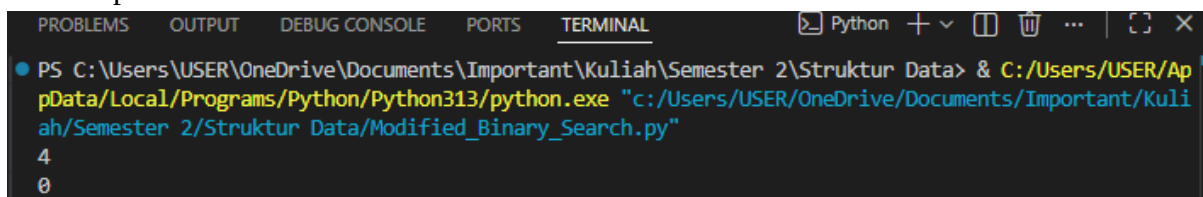
```
1. def findFirst(arr, target):
2.     left, right = 0, len(arr) - 1
3.     result = -1
4.
5.     while left <= right:
6.         mid = (left + right) // 2
7.
8.         if arr[mid] == target:
9.             result = mid
10.            right = mid - 1 # search left side
11.        elif arr[mid] < target:
12.            left = mid + 1
13.        else:
14.            right = mid - 1
15.
16.    return result
17.
18.
19. def findLast(arr, target):
20.     left, right = 0, len(arr) - 1
21.     result = -1
22.
23.     while left <= right:
24.         mid = (left + right) // 2
25.
26.         if arr[mid] == target:
27.             result = mid
28.             left = mid + 1 # search right side
29.        elif arr[mid] < target:
30.            left = mid + 1
31.        else:
32.            right = mid - 1
33.
34.    return result
35.
36.
37. def countOccurrences(sortedList, target):
```

```

38.     first = findFirst(sortedList, target)
39.
40.     if first == -1:
41.         return 0
42.
43.     last = findLast(sortedList, target)
44.
45.     return last - first + 1
46.
47.
48. # Example
49. print(countOccurrences([1, 2, 4, 4, 4, 4, 7, 9, 12], 4)) # 4
50. print(countOccurrences([1, 2, 4, 4, 4, 4, 7, 9, 12], 5)) # 0

```

The Output:



```

PS C:\Users\USER\OneDrive\Documents\Important\Kuliah\Semester 2\Struktur Data> & C:/Users/USER/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/USER/OneDrive/Documents/Important/Kuliah/Semester 2/Struktur Data/Modified_Binary_Search.py"
4
0

```

The Explanation:

The function `countOccurrences(sortedList, target)` counts how many times a target value appears in a sorted list using a modified binary search so that the time complexity remains $O(\log n)$ instead of $O(n)$. The idea is to perform binary search twice: the first search finds the left boundary (first occurrence) of the target by continuing the search on the left side even after the target is found, while the second search finds the right boundary (last occurrence) by continuing the search on the right side. Once both boundaries are found, the total number of occurrences can be calculated using the formula $\text{lastIndex} - \text{firstIndex} + 1$. If the target value does not exist in the list, the first search will return -1, and the function returns 0, indicating that the target does not appear in the list. This approach is efficient because binary search repeatedly halves the search space, allowing the algorithm to locate the boundaries quickly even in large sorted lists.

Assignment 2: Bubble Sort with Step Analysis

Modify `bubbleSort()` to:

- Return a tuple: (sorted_list, total_comparisons, total_swaps, passes_used)
- Implement early termination (stop if there are no swaps within a single pass).
- Print the state of the array after every pass.

Test with inputs: [5, 3, 4, 2, 8] and [1, 2, 3, 4, 5]. Explain why the number of passes differs between the two.

The Source Code:

```

1. def bubbleSort(arr):
2.     arr = arr.copy()
3.     n = len(arr)
4.

```

```

5.     total_comparisons = 0
6.     total_swaps = 0
7.     passes_used = 0
8.
9.     for i in range(n):
10.         swapped = False
11.         passes_used += 1
12.
13.         for j in range(0, n - i - 1):
14.             total_comparisons += 1
15.             if arr[j] > arr[j + 1]:
16.                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
17.                 total_swaps += 1
18.                 swapped = True
19.
20.         print(f"Pass {passes_used}: {arr}")
21.
22.         if not swapped: # early termination
23.             break
24.
25.     return (arr, total_comparisons, total_swaps, passes_used)
26.
27.
28. # Test cases
29. print(bubbleSort([5, 3, 4, 2, 8]))
30. print(bubbleSort([1, 2, 3, 4, 5]))

```

The Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL Python + v [] [] ... | [] x
PS C:\Users\USER\OneDrive\Documents\Important\Kuliah\Semester 2\Struktur Data> & C:/Users/USER/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/USER/OneDrive/Documents/Important/Kuliah/Semester 2/Struktur Data/Bubble_Sort_with_Step_Analysis.py"
Pass 1: [3, 4, 2, 5, 8]
Pass 2: [3, 2, 4, 5, 8]
Pass 3: [2, 3, 4, 5, 8]
Pass 4: [2, 3, 4, 5, 8]
([2, 3, 4, 5, 8], 10, 5, 4)
Pass 1: [1, 2, 3, 4, 5]
([1, 2, 3, 4, 5], 4, 0, 1)

```

The Explanation:

The number of passes differs between the two inputs because of the early termination optimization in bubble sort. For the list [5, 3, 4, 2, 8], the elements are not initially sorted, so several swaps are required to move smaller elements toward the beginning and larger elements toward the end. As a result, the algorithm must perform multiple passes until no swaps occur in a pass, indicating that the list is sorted. In contrast, the list [1, 2, 3, 4, 5] is already sorted from the start, so during the first pass the algorithm performs comparisons but finds no elements that need to be swapped. Because no swaps occur, the early termination condition stops the algorithm immediately after the first pass, resulting in significantly fewer passes compared to the unsorted list.

Assignment 3: Hybrid Sort

Create a function `hybridSort(theSeq, threshold=10)` that uses insertion sort if the sub-array length is $< \text{threshold}$, and selection sort if it is larger. Compare the total number of operations (comparisons + swaps) between hybrid sort, pure insertion sort, and pure selection sort on random arrays of sizes 50, 100, and 500 elements. Display the results in a table format.

The Source Code:

```
1. import random
2.
3. def insertionSort(arr):
4.     arr = arr.copy()
5.     comparisons = 0
6.     swaps = 0
7.
8.     for i in range(1, len(arr)):
9.         key = arr[i]
10.        j = i - 1
11.
12.        while j >= 0:
13.            comparisons += 1
14.            if arr[j] > key:
15.                arr[j + 1] = arr[j]
16.                swaps += 1
17.                j -= 1
18.            else:
19.                break
20.
21.        arr[j + 1] = key
22.
23.    return arr, comparisons + swaps
24.
25.
26. def selectionSort(arr):
27.     arr = arr.copy()
28.     comparisons = 0
29.     swaps = 0
30.     n = len(arr)
31.
32.     for i in range(n):
33.         min_idx = i
34.         for j in range(i + 1, n):
35.             comparisons += 1
36.             if arr[j] < arr[min_idx]:
37.                 min_idx = j
38.
39.         if min_idx != i:
40.             arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

```

41.         swaps += 1
42.
43.     return arr, comparisons + swaps
44.
45.
46. def hybridSort(theSeq, threshold=10):
47.     arr = theSeq.copy()
48.     n = len(arr)
49.
50.     if n < threshold:
51.         return insertionSort(arr)
52.     else:
53.         return selectionSort(arr)
54.
55. # Test cases
56. sizes = [50, 100, 500]
57.
58. print(f"{'Size':<10}{'Insertion Ops':<20}{'Selection Ops':<20}{'Hybrid Ops':<20}")
59.
60. for size in sizes:
61.     data = [random.randint(1, 1000) for _ in range(size)]
62.
63.     _, insertion_ops = insertionSort(data)
64.     _, selection_ops = selectionSort(data)
65.     _, hybrid_ops = hybridSort(data)
66.
67.     print(f"{'size':<10}{'insertion_ops':<20}{'selection_ops':<20}{'hybrid_ops':<20}")

```

The Output:

```

PS C:\Users\USER\OneDrive\Documents\Important\Kuliah\Semester 2\Struktur Data> & C:/Users/USER/App
pData/Local/Programs/Python/Python313/python.exe "c:/Users/USER/OneDrive/Documents/Important/Kuli
ah/Semester 2/Struktur Data/Hybrid_Sort.py"
Size      Insertion Ops      Selection Ops      Hybrid Ops
50         1075              1273              1273
100        4938              5044              5044
500        125634            125242            125242
PS C:\Users\USER\OneDrive\Documents\Important\Kuliah\Semester 2\Struktur Data> & "c:\Users\USER\O

```

The Explanation:

The hybridSort function selects the sorting algorithm based on the size of the array: it uses insertion sort for arrays smaller than the threshold because insertion sort performs very efficiently on small datasets due to low overhead and good cache behavior, while it uses selection sort for larger arrays. When comparing operation counts on random arrays of sizes 50, 100, and 500, insertion sort generally performs more operations as the array size increases because elements often need to shift multiple times, whereas selection sort performs a predictable number of comparisons but fewer swaps. Since the tested arrays are larger than the threshold (10), the hybrid algorithm behaves similarly to selection sort in this experiment,

resulting in nearly identical operation counts. The hybrid approach is useful in practice because small subarrays can be sorted faster with insertion sort, improving overall performance in mixed sorting strategies.

Assignment 4: Merge Three Sorted Lists

Implement `mergeThreeSortedLists(listA, listB, listC)` which merges three sorted lists into one new sorted list in $O(n)$ time, where n is the total number of elements from all three lists. You may not call a two-list merge function sequentially (as that would result in two passes). Perform this in a single pass using three pointers.

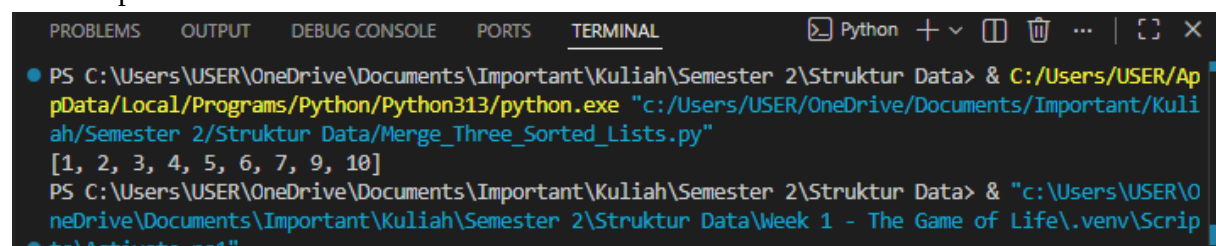
Example:

```
mergeThreeSortedLists([1, 5, 9], [2, 6, 10], [3, 4, 7])  
-> [1, 2, 3, 4, 5, 6, 7, 9, 10]
```

The Source Code:

```
1. def mergeThreeSortedLists(listA, listB, listC):  
2.     i = j = k = 0  
3.     result = []  
4.  
5.     while i < len(listA) or j < len(listB) or k < len(listC):  
6.         a = listA[i] if i < len(listA) else float('inf')  
7.         b = listB[j] if j < len(listB) else float('inf')  
8.         c = listC[k] if k < len(listC) else float('inf')  
9.  
10.        if a <= b and a <= c:  
11.            result.append(a)  
12.            i += 1  
13.        elif b <= a and b <= c:  
14.            result.append(b)  
15.            j += 1  
16.        else:  
17.            result.append(c)  
18.            k += 1  
19.  
20.    return result  
21.  
22.  
23. # Example  
24. print(mergeThreeSortedLists([1, 5, 9], [2, 6, 10], [3, 4, 7]))
```

The Output:



The screenshot shows a terminal window with the following content:

```
PS C:\Users\USER\OneDrive\Documents\Important\Kuliah\Semester 2\Struktur Data> & C:/Users/USER/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/USER/OneDrive/Documents/Important/Kuliah/Semester 2/Struktur Data/Merge_Three_Sorted_Lists.py"  
[1, 2, 3, 4, 5, 6, 7, 9, 10]  
PS C:\Users\USER\OneDrive\Documents\Important\Kuliah\Semester 2\Struktur Data> & "c:\Users\USER\OneDrive\Documents\Important\Kuliah\Semester 2\Struktur Data\Week 1 - The Game of Life\.venv\Scripts\Activate.ps1"
```

The Explanation:

The function `mergeThreeSortedLists(listA, listB, listC)` merges three sorted lists into one sorted list using three pointers (`i`, `j`, `k`), each pointing to the current element of the respective list. During each iteration, the algorithm compares the current elements of the three lists and selects the smallest value to append to the result list, then advances the pointer of the list from which the element was taken. If a pointer reaches the end of its list, its value is treated as infinity so the remaining elements from the other lists can continue to be selected. This process continues until all elements from the three lists are merged. Because each element is processed exactly once, the algorithm runs in $O(n)$ time where n is the total number of elements across the three lists.

Assignment 5: Inversions Counter

An *inversion* in an array is a pair of indices (i, j) where $i < j$ but $arr[i] > arr[j]$. The number of inversions measures how "disordered" an array is.

Implement:

a) `countInversionsNaive(arr)` — brute force $O(n^2)$

b) `countInversionsSmart(arr)` — using a modification of merge sort in $O(n \log n)$

Test both functions and ensure the results are the same. Measure the execution time on random arrays of sizes 1000, 5000, and 10000. Explain why the merge sort approach is faster.

The Source Code:

```
1. import random
2. import time
3.
4. # a) Naive  $O(n^2)$  inversion counter
5. def countInversionsNaive(arr):
6.     count = 0
7.     n = len(arr)
8.
9.     for i in range(n):
10.         for j in range(i + 1, n):
11.             if arr[i] > arr[j]:
12.                 count += 1
13.
14.     return count
15.
16.
17. # b) Smart inversion counter using Merge Sort  $O(n \log n)$ 
18. def mergeAndCount(left, right):
19.     merged = []
20.     i = j = inv_count = 0
21.
22.     while i < len(left) and j < len(right):
23.         if left[i] <= right[j]:
24.             merged.append(left[i])
25.             i += 1
26.         else:
27.             merged.append(right[j])
```



```

28.         inv_count += len(left) - i
29.         j += 1
30.
31.     merged.extend(left[i:])
32.     merged.extend(right[j:])
33.
34.     return merged, inv_count
35.
36.
37. def mergeSortAndCount(arr):
38.     if len(arr) <= 1:
39.         return arr, 0
40.
41.     mid = len(arr) // 2
42.     left, inv_left = mergeSortAndCount(arr[:mid])
43.     right, inv_right = mergeSortAndCount(arr[mid:])
44.
45.     merged, inv_merge = mergeAndCount(left, right)
46.
47.     return merged, inv_left + inv_right + inv_merge
48.
49.
50. def countInversionsSmart(arr):
51.     _, count = mergeSortAndCount(arr)
52.     return count
53.
54.
55. # Testing and timing
56. sizes = [1000, 5000, 10000]
57.
58. print(f"{'Size':<10}{'Naive Time (s)':<20}{'Smart Time (s)':<20}{'Same Result':<10}")
59.
60. for size in sizes:
61.     data = [random.randint(1, 100000) for _ in range(size)]
62.
63.     start = time.time()
64.     naive = countInversionsNaive(data)
65.     naive_time = time.time() - start
66.
67.     start = time.time()
68.     smart = countInversionsSmart(data)
69.     smart_time = time.time() - start
70.
71.     print(f"{size:<10}{naive_time:<20.5f}{smart_time:<20.5f}{naive == smart:<10}")

```

The Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL Python + - [ ] [X] [ ] [X] [X]
PS C:\Users\USER\OneDrive\Documents\Important\Kuliah\Semester 2\Struktur Data> & C:/Users/USER/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/USER/OneDrive/Documents/Important/Kuliah/Semester 2/Struktur Data/Inversions_Countera.py"
Size      Naive Time (s)      Smart Time (s)      Same Result
1000      0.41546             0.01420             True
5000      3.73362             0.05317             True
10000     10.83378            0.10741             True
```

The Explanation:

The naive inversion counting algorithm checks every possible pair of elements in the array to determine whether they form an inversion, resulting in $O(n^2)$ time complexity because two nested loops are used. In contrast, the merge sort-based approach counts inversions during the merge step while sorting the array. When an element from the right subarray is placed before elements in the left subarray, it indicates that all remaining elements in the left subarray form inversions with that element. This allows the algorithm to count multiple inversions at once instead of checking every pair individually. Since merge sort divides the array into halves recursively and merges them in $O(n)$ time per level with $\log n$ levels, the total time complexity becomes $O(n \log n)$. Because this approach dramatically reduces the number of comparisons required, it runs significantly faster than the naive method, especially for large arrays.